

ICT703 Programming

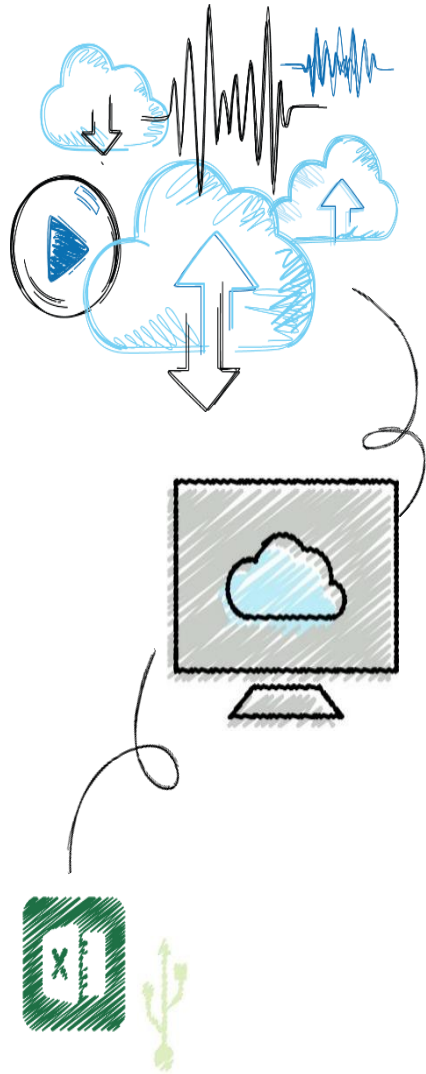
Module 9

Classes and Objects



Objectives

- Review Object Orientated Programming
- Classes vs Objects
- Attributes and Methods





Benefits of OOP (Object-Oriented Programming)?

- It is the most common style of programming used these days
 - In Python, Java, C#, C++, JavaScript, Kotlin, Ruby, Ada, ...
- Benefits include:
 - encapsulation: we can protect data inside an object (modularity, changeability)
 - polymorphism: we can write extensible programs that use many kinds of objects (even ones that have not been written yet!)
 - inheritance: we can define new classes by extending a similar class
- GUIs: most Graphical User Interface libraries use OOP



Classes are the Building Blocks

Python classes allow us to create custom objects

- each **class** defines a set of similar **objects**. *It is a template*
- we can specify what *data fields (attributes)* those objects will have
- we can specify what *methods (functions)* they will all have
- so **classes** allow us to group related *data and methods* together



We have used many kinds of objects

- List objects
 - methods: `append(val)`, `remove(val)`, ...
- File objects:
 - methods: `write(...)`, `readline()` ...

What about creating our own?



Defining our own objects - 1

- We are going to create a program about tennis players. Let's start with the simplest definition of a class that has 1 attribute and 1 method.

```
class player:
```

```
    name = "Andrew"
```

```
    def print_name(self):
```

```
        print(self.name)
```

```
p1 = player()
```

```
p1.print_name()
```



Defining our own objects - 2

```
class player:
```

```
    def __init__(self, name, wins, loss):
```

```
        self.name = name
```

```
        self.wins = wins
```

```
        self.loss = loss
```

```
    def win(self):
```

```
        self.wins += 1
```

```
    def loses(self):
```

```
        self.loss += 1
```

```
p1 = player("Eve",3,1)
```

```
p2 = player("Andrew", 5, 2)
```

```
print("Player 1 has a percentage of: ", p1.wins/p1.loss)
```

```
p1.loses()
```

```
p2.win()
```

```
print("Player 1 has a percentage of: ", p1.wins/p1.loss)
```



Note the Syntax

- We use just a dot to refer to attributes
 - (write) (read)
 - `p1.wins = p1.wins + 1`
- We use a dot plus brackets to call a method
 - `p1.birthday()`
 - `stats = p1.getStats()`